

AUDITORÍA TÉCNICA

SalesVoice

Diagnóstico de las caídas de llamada y plan de corrección por fases



Práctica de ventas por voz con IA en tiempo real

Fecha: 20 de julio de 2026

Repositorio auditado: tec-blip/voice (rama main)

Base de datos: Supabase «Voice Sales»

Método: análisis de código + límites oficiales de plataforma + **forense de las llamadas reales de la base de datos**

1 Explicación sencilla



Imagina que la app es un **teléfono mágico** donde practicas ventas hablando con un **robot que hace de cliente**. El problema que reportan los usuarios: **la llamada se corta sola** — a veces al minuto, a veces a los 5, a los 10... y otras veces funciona bien. Por eso parece un misterio.

Descubrimos que **no es un solo problema, son tres**, y por eso parece aleatorio:

- 1 **El robot cuelga solo.** Si te quedas callado un momento para pensar, el robot cree que "ya terminaron" y cuelga — como un amigo impaciente. **Es la causa más frecuente hoy.**
- 2 **Cada 10 minutos la línea con Google se reinicia sola.** La app intenta reconectar sin que lo notes, pero a veces falla y la llamada se pierde entera. Pasados ~15 min hay un segundo muro que nadie ha cruzado todavía.
- 3 **A veces la llamada ni siquiera arranca bien** la primera vez, y no lo reintenta solo.



La solución: enseñarle al teléfono a **(1)** no colgar solo aunque haya silencios, **(2)** reconectarse siempre sin que se note y aguantar llamadas largas, y **(3)** reintentar si algo falla al empezar.

2 Resumen ejecutivo

✓ Lo que está bien

- Arquitectura sólida: lógica determinista separada de la IA, con tests.
- Puntuación recalculada en el servidor; seguridad por capas.
- La concurrencia NO es el problema: la infra aguanta ~1.000 sesiones simultáneas.
- Migraciones sincronizadas con la base de datos (sin drift).

✗ Lo que falla

- Caídas intermitentes de llamada con **3 causas solapadas**.
- Ninguna llamada en la historia pasó de **20.2 min**.
- "45 min continuos" hoy es **imposible** (2 topes duros).
- Fixes previos atacaban 1 causa a la vez, nunca las tres.

Diagnóstico central: el problema no es la arquitectura general — es que las caídas tienen tres orígenes distintos (uno de lógica propia, uno de plataforma, uno de arranque) que se manifiestan en momentos diferentes. Resolverlos requiere abordar los tres, no uno.

3 Qué está mal — hallazgos por severidad

● CRÍTICO — rompe la llamada

#	PROBLEMA	EVIDENCIA REAL (DB)
C1	El robot cuelga solo con <code>timeout</code> en cualquier minuto. El filtro bloquea el cierre triunfalista pero deja pasar <code>timeout</code> pasado el piso (30–90 s). El modelo lo sobre-usa ante silencios. Para el alumno = caída.	4 de 8 llamadas terminaron así, a los 1.4, 6.7, 6.9 y 15.3 min.
C2	Muro de ~10 min (reconexión frágil) + muro de ~15 min (sin compresión de contexto). Sin <code>contextWindowCompression</code> , Google corta el audio a ~15 min por límite de tokens.	El 3-jul murieron 3 llamadas de ~9.6 min sin guardarse. Récord: 15.3 min.
C3	Tope duro en 45:00 exactos → auto-cuelgue. Una prueba "de 45 min" se corta sola al llegar. <code>MAX_CALL_SECONDS = 2700</code>	—

● ALTO — fallos frecuentes en llamadas largas / móvil

#	PROBLEMA	IMPACTO
A1	Carrera de sockets duplicados en la reconexión (sin mutex): red inestable + timer del <code>goAway</code> pueden abrir 2 WebSockets con el mismo handle.	Audio doble, transcript duplicado, handle invalidado.
A2	En móvil, si muere el micrófono la llamada se descarta sin evaluación ni transcript.	El alumno pierde toda la práctica en el minuto 40.
A3	Fallos de arranque sin reintento (minuto 0–1).	"No funcionó, reintenté y ya."

● MEDIO — costo / seguridad / futuro (no rompen la llamada)

#	PROBLEMA
M1	Token GCP con scope <code>cloud-platform</code> completo entregado al navegador en la query string (riesgo de seguridad, puede filtrarse a logs).
M2	Pre-fetch de token del <code>goAway</code> es código muerto → duplica el minado de tokens sin beneficio.
M3	Cuota reservada (600 s) subestima llamadas de 45 min → un usuario podría quemar ~1 h 45 min/día.
M4	Cuotas hardcodeadas (<code>MAX_CONCURRENT=1</code> , <code>DAILY_LIMIT=3600</code>) → cada prueba simultánea exige cuenta distinta; 1 prueba de 45 min por usuario/día.
M5	<code>/api/evaluate</code> sin <code>maxDuration</code> ; N Llamadas terminando a la vez → posibles 429 si la API key es free tier.
M6	"Leaked password protection" desactivada en Supabase Auth (un clic en el dashboard).

4 La prueba: quién colgó realmente

La app guarda la "caja negra" de cada llamada (eventos del ciclo de vida). En las 8 llamadas instrumentadas, **el modelo (robot) fue quien colgó con timeout** en la mitad — a minutos completamente distintos. Eso explica el "a veces al minuto, a veces a los 10":

Llamada tipo «cierre» — el robot colgó con timeout	1.4 min
Llamada tipo «general» — el robot colgó con timeout	6.7 min
Llamada tipo «general» — el robot colgó con timeout	6.9 min
Llamada tipo «general» — el robot colgó con timeout	15.3 min
3 de julio — 3 llamadas murieron en el muro de los ~10 min	~9.6 min • perdidas

Es **intermitente por naturaleza**: depende del juicio del modelo sobre la conversación — justo el tipo de decisión que la propia filosofía del proyecto dice que el LLM NO debe tomar. Por eso no se puede arreglar solo con el prompt; hay que bloquearlo en el código determinista.

Fase 0 • Verificar

~1 día

Confirmar datos externos que condicionan todo (no requiere código).

- ☐ Plan de Vercel (Hobby vs Pro) → define el `maxDuration` máximo.
- ☐ Tier de `GEMINI_API_KEY` (free vs pago) → define si el burst da 429.
- ☐ Cuota de sesiones Live concurrentes del proyecto GCP.
- ☐ Confirmar que Supabase no se auto-pausará durante las pruebas.

Fase 1 • Detener las caídas (CRÍTICO)

~1–2 días

Que la llamada deje de cortarse sola.

- ☐ **C1** Bloquear `timeout` del modelo en arco completo + test. La infraestructura de re-enganche ya existe: el robot seguirá hablando en vez de colgar.
- ☐ **C2** Añadir `contextWindowCompression: {slidingWindow:{}}` al setup + reprogramar el timer de reconexión.
- ☐ **C3** Subir el cap duro a 50 min (1 línea).

🔗 Prueba: 1 llamada de 20+ min que antes moría.

Fase 2 • Robustez y llamadas largas

~2–3 días

Que aguante 45 min continuos y en móvil.

- ☐ **A1** Mutex de reconexión (elimina sockets duplicados).
 - ☐ **A2** Guardar sesión con evaluación si muere el mic pasado el piso.
 - ☐ **A3** Reintento automático en el arranque.
 - ☐ **M2** Eliminar el pre-fetch muerto (o usarlo de verdad).
- 🔗 Prueba objetivo: 1 llamada continua de 45 min completa (desktop → móvil).

Fase 3 • Simultáneas + costo/seguridad

~2 días

Varias llamadas de 45 min a la vez.

- ☐ **M4** Cuotas a variables de entorno (subir para la tanda).
 - ☐ **M5** `maxDuration` en `/api/evaluate`.
 - ☐ **M3** Alinear reserva de cuota con la duración real.
 - ☐ **M1** Acotar scope/lifetime del token; sacarlo de la query string.
 - ☐ **M6** Activar leaked-password protection.
- 🔗 Prueba: N=5 cuentas, arranques escalonados ~1 min, 45 min c/u → subir a 10/20.

No urgente, pero importante que no se olvide.

- ☐ Migrar del modelo `gemini-live-2.5-flash-native-audio` antes del **2026-12-13** (retiro programado).
- ☐ Vigilar el crecimiento de la DB (transcripciones JSONB vs límite de 500 MB del plan).
- ☐ Limpieza periódica de `active_sessions` / `rate_limits` (crecen sin purga).

6 Temas abiertos — a verificar

No se pueden determinar desde el código y podrían ser importantes más adelante:

- 1 Cuota de sesiones Live concurrentes de GCP** — si Vertex rechaza sesiones, la app lo muestra como caída genérica. Candidato #1 a romper una tanda de 10+ llamadas.
- 2 Tier de la API key de evaluación** — en free tier (~10 RPM), varias evaluaciones simultáneas darán 429.
- 3 Plan de Vercel** — determina el `maxDuration` máximo de los endpoints.
- 4 Auto-pausa de Supabase Free** — a los 7 días de inactividad (le pasó al proyecto gemelo). Una demo un lunes puede encontrar la DB dormida.

⚠ **Advertencia crítica:** existe un repo/fork abandonado `tec-blip/shiav2` (rama `v2-gamification`) con un esquema de base de datos **incompatible**. NO desplegar ese por error: no coincide con la DB de producción. El repo bueno es `tec-blip/voice`.

7 Lo que SÍ funciona (no re-arreglar)

- ✓ Session resumption y refresh de token: correctos (el token GCP vive 1 h, no 2 min).
- ✓ El contador de reintentos se resetea bien por ciclo; no se acumula.
- ✓ No hay fugas de memoria significativas en llamadas largas.
- ✓ La reserva de 600 s NO corta la llamada; el heartbeat la mantiene viva.
- ✓ Trigger de rankings en producción es la versión ligera (verificado en la DB).
- ✓ Migraciones 001–007 aplicadas y sincronizadas (sin drift).